Computational Evolution of Mobility

Andrew S. Cantino
33 Cable Lane, Athens, OH 45701 USA
Athens High School, The Plains, OH 45780 USA
12th Grade

TABLE OF CONTENTS

INTRODUCTION

The online site Sodaplay (http://www.sodaplay.com) makes it possible to create life-like constructs made of muscles, masses, and springs. It seemed plausible that the application of evolutionary algorithms to a Sodaplay-like system could allow the evolution of mobile "creatures." Toward this end, a computational physics model capable of simulating the interactions among masses, springs, and "muscles" (waveform-powered springs) in a two dimensional, gravitational environment was created as a framework on which to run an evolutionary algorithm. It was hypothesized that, with a well-designed physics model, evolutionary algorithms could generate mobile "creatures" that utilize various modes of locomotion. Before applying the evolutionary algorithm, the response of the physics model to various scenarios was tested. After the model was tested, multiple evolutionary runs were performed. A great variety of "creatures" evolved, including those that run, jump, walk, and slide.

EXPERIMENT

The first step of this project was the research and development of a physics model capable of simulating the interactions of masses and springs in a gravitational environment. In addition, the physics model required sufficient robustness to simulate the unpredictable outcomes of the evolutionary algorithm (evolutionary algorithms will be discussed later).

The first attempt at such a physics model was written in a programming language called REALbasic (http://www.realbasic.com). REALbasic is a Macintosh rendition of Visual Basic and is especially useful for rapid prototype development. In the case of this project, its rapid development abilities allowed time to be spent developing the algorithms for the physics model itself instead of interface building and manipulation. REALbasic was used for all completed versions of this project, although development has been started on a C++ version, which will be discussed later.

The earliest physics model used the muscle as its fundamental object. The computer stored an array of muscles and points in its memory. When the program was

run, the screen displayed a number of dots (masses) connected by lines (muscles) in two-dimensional space. Different muscles could be told to expand or contract, shifting the points connected to them by varying amounts. If multiple muscles were connected to the same point, then complicated dynamics could occur when the change in one muscle moved points connected to other muscles. This was a step in the right direction, but it was hard to predict and control.

Although the original physics model was deemed unsuitable for the project and scrapped, some important lessons were learned from it. First, the results of any simulation run by the physics model would be hard to interpret logically because of the number of variables involved in the dynamic system. Secondly, it was determined that a better way of dealing with the motion of the masses and the effects of the springs was needed to model the complicated systems that evolution would probably create. Finally, mathematical code was developed that would be reused later to shift points a given distance along a line in the computer's display grid system.

Given the expected difficulty of adding fundamental physical laws to the original system (where even gravity was nonexistent), a new method of dealing with forces was needed. Vectors seemed to be the logical choice, so research was done to that end, and a new physics model was created. Vectors allowed forces from linear springs and gravity to be applied with ease. Still, many problems were encountered. One of the earliest and hardest to overcome was the translation of angles to slopes and visa versa. The problem was that a slope as a single number contains less information than an angle, because a slope can go in two directions at once. This was eventually resolved by keeping the y and x displacements separate whenever possible.

The new physics model has been modified greatly, but it is still fundamentally similar to the one created at this stage of the project. The model consists of masses stored in the computer's memory. The masses have (along with their mass) properties such as velocity, position, and momentum. The computer also stores an array of springs. Springs are line segments that have a mass at each end, and they apply a

Hooke's Law: (see Serway, p. 345)

$F_S = -kx$

where F_S is the force applied by the spring, k is the spring constant, and x is the current displacement of the spring.

vector force to their associated masses in accordance with Hooke's Law. In addition to masses and springs, the physics model includes the ability to store and add cosine waveforms. These waveforms are added together and used to control muscles (as will be described). The waveforms have a constantly increasing phase displacement, resulting in the visual effect of the waveform scrolling by on the screen. The muscles are modified springs defined by a stationary position under which the waveform scrolls. As the waveform passes this position, the value of the waveform (visually, the height above or below the x-axis) defines the rest-length of the spring and thus the action of the muscle.

In addition to masses, muscles, and springs, the physics model includes friction with the floor, gravity, internal friction, and semi-elastic collisions between the masses and the floor. Like the original physics engine, the current engine uses an iteration-based approach to modeling the complex interactions in the system. Given small time increments (for experimentation, .1 second was used), the engine calculates force vectors for each mass (based on gravity, spring forces, and, in a slightly different manner, friction) and then approximates the movement of the various masses during the given time step. This cycle repeats until the engine is stopped.

Operation of the physics model

- Calculate force vectors for all masses. These vectors represent the force of gravity and the pull of springs.
- Calculate vector sum for all masses.
- Apply friction to any mass in contact with the floor. Friction is applied as a force parallel to the floor and opposite the motion of the mass. It is equal to a frictional constant multiplied times the normal force to the floor. (In the flat-floor situation of the experimental model, the normal is the same as the y-component of the force vector sum for any given mass. As will be discussed later, a new physics engine in progress has rough-floors with surface features and has a much more advanced friction algorithm.)
- For every mass, calculate the change in its current velocity due to the application of the force vector sum. (For actual equation, see Appendix A.)
- For every mass, use its velocity and the iteration time increment to calculate a new position. Move to that position.
- Check for possible collision with the floor and, if found, calculate reflection.

After the basic physics model was finished, testing was needed to verify its operation. It was deemed impractical to test situations in the physics engine by testing for accuracy of real-life prediction. Instead, a number of test situations were used to

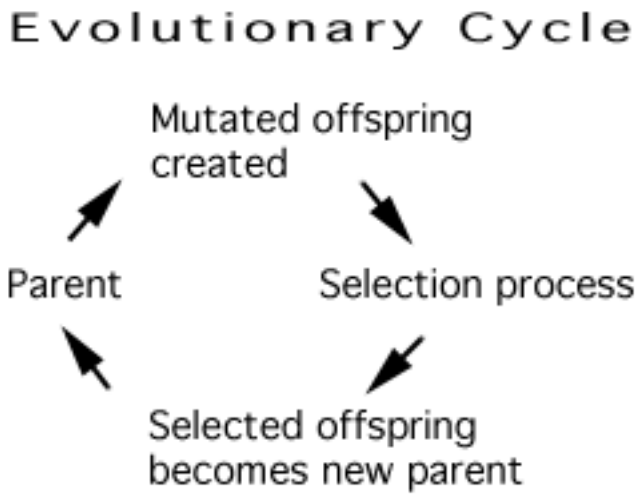
check for visual and intuitive correctness (see explanation box). In all cases, the engine's results made intuitive sense and "felt" right. Because the physics engine is designed to be the framework for an evolutionary algorithm, it needs to follow rules systematically, but it does not necessarily need to predict real-life interactions exactly. The physics engine becomes the universe for the evolution to occur in. It has to be static and predictable, but it does not need to be exactly the same as our universe. In fact, this opens up the possibility of evolving "creatures" in varying conditions, such as high or low gravity environments.

Once the physics model was finished, the evolutionary algorithm was written. Evolutionary algorithms use the principles of Darwinian evolution inside a computer system to evolve solutions to problems. "Although simplistic from a biologist's viewpoint, these algorithms are sufficiently complex to provide robust and powerful adaptive search mechanisms" (Heitkoetter Q1).

Tests applied to the physics model

The physics model was tested with the following cases in order to verify correct operation:

- Single joint dropped (should bounce and come to rest)
- Pair of joints connected by a spring horizontally and dropped (should bounce in unison and come to rest)
- Pair of joints connected by a spring vertically and dropped (should perform a double bounce)
- Test waveform addition and powered spring performance
- Look for "seemingly accurate" simulation of complex spring constructs such as boxes, thrown weights, and mobile "creatures" using muscles.



The evolutionary algorithm consists of a repeated generational cycle in which a parent creature has 15 offspring (each initially identical to the parent creature.) Then, each offspring has a random mutation applied to it (for a list of mutations, see Appendix B), and is run through the physics model for a certain number of iterations (for experimentation, 2000 iterations of the physics engine were run per offspring). After the offspring are run through the physics engine, the offspring whose center of mass displaced the farthest is selected to become the new parent, and the cycle repeats. Other selection criteria can be used, but for the experimentation in this project, displacement was used. Some test runs were also performed with center of mass vertical height used as a selection criterion instead of displacement. Some "creatures" evolved that jumped, others stayed stationary and took tall, tower-like forms.

DISCUSSION

Once the program was complete, evolutionary runs were performed. The locomotive methods that evolved during these runs were extremely varied. All runs started from the same initial condition (see Appendix C) but quickly diverged to utilize many different types of locomotion. Some "creatures" ran, others slid, hopped, walked, or even (apparently) galloped or rolled. Many "creatures" used a form of hopping or jumping to travel. The hypothesis that, with a well-designed physics model, evolutionary algorithms could generate mobile "creatures" that utilize various modes of locomotion was thus confirmed.

The evolutionary algorithm used in this project is a very simple one, but it resembles in many ways the optimization technique knows as Evolution Strategy. Evolution Strategy is a type of evolutionary algorithm that was developed in the 1960's by Ingo Rechenberg and Hans-Paul Schwefel at the Technical University of Berlin (Heitkoetter Q1.3). Like Evolution Strategy, the algorithm used in this project used mutation to modify a parent solution and then used a rating system to select the best offspring. However, Evolution Strategy is different in some ways and is more advanced than the system used in this project.

A large amount of work has previously been done in the field of evolutionary algorithms. Evolutionary algorithms have been used to solve many types of optimization problems. Recently, Hod Lipson and Jordan B. Pollack at Brandeis University have done work involving the generation via evolutionary algorithms of mobile robots. The robots were evolved inside of a computer physics simulator and then fabricated using "off-the-shelf rapid manufacturing technology" (Lipson, p.1). Research is also being performed at the Iowa State Visualization Lab where virtual robots are being evolved using a Genetic Algorithm, a subset of evolutionary algorithms that evolves through changes in a simulated genetic code (Oliver, p.1).

The simplicity of the evolutionary algorithm used in this project can, in some ways, be seen as beneficial. The fact that such a simple algorithm could generate such a wide variety of effective solutions to the locomotion problem is interesting and has implications for the evolution of solutions to other problems. With even a slightly more advanced algorithm, many problem-solving possibilities emerge.

Besides running evolutionary tests to see what would evolve, experimentation was performed with the evolutionary algorithm. Multiple runs were performed, all for the same number of generations and physics engine iterations. The displacement and complexity (complexity is a function of to the number of muscles, masses, and waveforms in a "creature") for each generation of each run were recorded. It was found that, as the complexity of a creature increased, the displacement decreased (see Appendix D). This result concurred with visual observations: that evolutionary runs would sometimes result in huge, chaotic "creatures" that failed to locomote effectively. It was also found that the successfulness of the test runs varied greatly from run to run (see

Appendix D). Some runs created very effective "creatures" that traveled quickly and smoothly, while others created slower, less efficient "creatures." Some "creatures" hardly moved at all, even after many times the usual number of evolutionary generations.

From these results, it can be concluded that evolutionary algorithms, while effective at generating solutions, fail to guarantee an optimal answer. Because many runs result in less than optimal solutions, multiple runs must be performed to find a useful and effective solution. However, this is not a large problem because "creatures" evolve in surprisingly short time spans. Many "creatures" evolved to locomote in fifty or fewer generations. In addition, it was found that evolutionary algorithms allow the evolution of diverse solutions, all of which started from the same initial condition.

Another conclusion highlights one of the inherent weaknesses of the evolutionary algorithm used in this project: the evolution for a specific niche. In biology, a niche is the position and role of an organism within an ecological community (Solomon, p.1148). In the terms of the physics model, this meant that "creatures" evolved in one situation probably would not be effective in a slightly differing situation. For example, most "creatures" could not locomote backwards when the waveform was reversed.

In writing the physics model, many different problems were encountered. Numerous tests were needed to find and iron out bugs in the code. However, once the physics engine was well tested, it could be used for many different purposes. For example, as well as using the physics engine as a basis for the evolutionary process, some test situations were created that demonstrated interesting physical principles such as the apparent levitation of a spring when it double bounces (Graham, p.90). Furthermore, modifications to the physics engine could allow the evolution of many different types of solutions, including more robust "creatures" that could navigate rough terrain.

Such modifications are currently being worked on. An updated version of the physics model that includes floor surface features (and the reflection and friction needed to deal with those features) is nearing completion. A few lingering glitches remain, but the core code is complete and progress is being made. Additionally, a C++ version of the physics engine has been started. The C++ version, when complete, should be much faster than the REALbasic version. Only with the speed that C++ can provide will the project be able to advance to the next level. Some ideas for continuation in C++ are as follows.

Once a C++ physics engine framework is complete, it should be possible to add a z coordinate to all of the physical calculations (vector objects, friction, etc.). With a z coordinate, it should be possible to evolve three-dimensional "creatures." The addition of floor surface features in the third dimension would allow the evolution of robust three-dimensional "creatures." Additionally, the surfaces of the three-dimensional "creatures" could be given elastic properties, which would allow the physical interaction of multiple creatures. To facilitate that interaction, simple neural-network brains could be added to allow stabilization of the muscles and, more importantly, control of the waveform (perhaps with multiple waveforms controlling different muscles, so that controlled motion could evolve.)

Finally, these three-dimensional "creatures" could be introduced into a simulated ecosystem and the evolutionary algorithm could be modified to allow all offspring potentially to survive. Instead of the algorithm specifically testing for a selection criterion, the ecosystem itself would provide the evolutionary pressure either by competition for "food" or by other selective necessities. At this point, the project bridges the gap between Computer Science and Biology, and enters the field of Artificial Life (Liekens).

On a different path, the current physics engine and evolutionary algorithm, or updated C++ ones, could be modified to simulate and evolve solutions to other engineering problems. For example, the current model may be modifiable to evolve bridges.

CONCLUSION

In summary, the physics model written for this project was able to act as an adequate framework for the evolutionary algorithm, because it included enough functions to create a simulated universe. Albeit a simplified one, this universe provided for enough fundamental physical laws (friction, gravity, and their effects on masses, springs, and "muscles") to allow the evolutionary process to occur. The hypothesis that, with a well-designed physics model, evolutionary algorithms could generate mobile "creatures" that utilize various modes of locomotion was confirmed. Indeed, a great variety of

"creatures" evolved from the same initial starting conditions, including those that ran, jumped, walked, and slid. Experimentation with the evolutionary algorithm showed that, while multiple runs were needed to find exceptionally effective "creatures," mobile creatures evolved almost every time. However, the necessity for multiple runs did not present a large problem, because "creatures" evolved in surprisingly short amounts of time.

The fact that a simple evolutionary algorithm coupled with a vector physics model could quickly create multiple solutions to a problem (in this case, center of mass displacement distance) has implications for the applicability of evolutionary algorithms in engineering design and problem solving. While research in this field has been done and is increasing in pace, the ease with which an evolutionary algorithm was applied to a physics model leads one to believe that, in the future, even more productive ways to use evolutionary algorithms will be found. Their potential use in the real-world design of engineering solutions seems extremely likely and possibly invaluable.

Finally, this project presents multiple opportunities for continuation. Work is already underway on a more advanced REALbasic version of the physics engine that can model rough floor terrain. In addition, a C++ version of the system has been started. It is hoped that a third-dimension will be added to the C++ version and that it will be possible to evolve three-dimensional creatures inside a simulated ecosystem as a second phase of this project.

ACKNOWLEDGMETS

Aid with physical equations was provided by Mr. Tom Stork, an Athens High School physics teacher. Thank you also to Dr. James Tong for his continued support of the Southeastern Ohio Regional Science and Engineering Fair 2001, which has provided financial support to bring this project to the Intel International Science and Engineering Fair 2001. I would also like to thank my parents for their ongoing encouragement and their time and effort in helping me edit this report.

REFERENCES

Liekens, A. "Artificial Life." 2000. In: A. Liekens (editor): *Artificial Life Online 2.0*. <http://alife.org/index.php?page=alife&contents=alife>

Graham, Mark. “Analysis of Slinky Levitation.” *The Physics Teacher* February 2001: 90-91.

Heitkoetter, J. and D. Beasley, eds. "The Hitch-Hiker's Guide to Evolutionary Computation: A list of Frequently Asked Questions (FAQ).” 1994. USENET: comp.ai.genetic. Available via anonymous FTP from <rtfm.mit.edu:/pub/usenet/news.answers/ai-faq/genetic/> About 90 pages.

Knorr, M. Evolution Strategy. 23 June 1999. 3 May 2001 <http://www2.informatik.uni-erlangen.de/IMMD-II/Lehre/SS99/NaturAlgorithmen/Presentations/05.EvolutionStrategies/ES/>

Lipson, H. and J. B. Pollack. The Golem Project. 22 Feb. 2001. 23 Feb. 2001 <http://golem03.cs-i.brandeis.edu/>

Oliver, J. H., D. A. Ashlock, and J. F. Walker. “Evolution of Virtual Robots.” 2001. 17 April 2001. <http://www.vrac.iastate.edu/research/simulation/simbot/>

Serway, R. A. and J. S. Faughn. 1989. *College Physics*, second edition. Saunders College Publishing, Philadelphia.

Sodaplay.com, <http://sodaplay.com>

Solomon, E P., L. R. Berg, D. W. Martin, and C. Villee. *Biology*. Fort Worth: Saunders College Publishing, 1993.

APPENDICES

APPENDIX A

Equation used to calculate new velocity of a mass based off old velocity and a force vector affecting it:

$$\text{Velocity}_{NEW} = \text{Velocity}_{OLD}\text{Friction}_{AIR} + \Delta t\left(\frac{\text{Force}}{\text{Mass}}\right)$$

Internally, this equation is divided into its vector components and calculated for the x and y components separately, then is recombined to form the new velocity vector. The equation was derived from f=ma with an absorption constant (Friction_{AIR}) added to represent air friction.

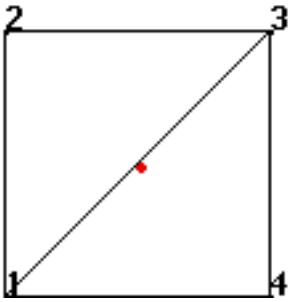
APPENDIX B

General mutations applied in the evolutionary process:

- Add joint with a spring to an existing joint
- Move joint
- Add spring
- Remove spring
- Add waveform
- Change waveform
- Add muscle
- Change the waveform phase position of a muscle
- Add a joint along an existing spring

APPENDIX C

This is a diagram of the configuration of the parent “creature” used during the evolutionary algorithm.



APPENDIX D

Experimental Results Data Table

Trial	**Displacement**	**Complexity**
1	533.702135	25
2	1640.71837	30
3	407.487742	41
4	538.181753	24
5	407.487742	41
6	1329.32419	30
7	1137.29407	53
8	777.772077	24
9	1469.77364	16
10	1207.45728	39
11	1664.35078	36
12	890.812062	31
13	307.801657	38
14	354.28545	166
15	818.968651	19
16	1113.33115	21
17	1364.10253	21
18	551.907026	85
19	1557.17812	34
20	328.468619	38
21	765.087498	28
22	2948.49592	24
23	668.123722	52
24	722.796017	23
25	845.20574	89
26	1854.67357	19

Displacement is the distance traveled by the “creatures.” Complexity represents the number of muscles, masses, and waveforms (assigned a heavier weighting).